

ACTIVIDAD: PLATAFORMA DE EVENTOS Y EXPERIENCIAS

Reto

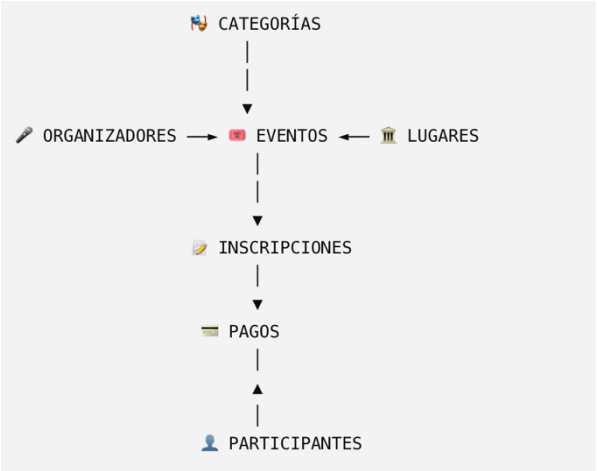
Una empresa organiza experiencias y eventos: conciertos, talleres, conferencias, festivales y actividades culturales.

La empresa necesita una base de datos para controlar **eventos, lugares, organizadores, participantes, categorías, inscripciones y pagos**.

El reto será construir la base de datos en **SQL Server** y posteriormente crear procedimientos almacenados capaces de **recibir información y devolver resultados**.

La idea que debe quedarles

Podemos plantearlo así:



Tenemos **7 tablas**:

#	Tabla	¿Qué almacena?
1	Categorias	Tipo de evento
2	Lugares	Sitios donde se realizan
3	Organizadores	Personas que organizan
4	Eventos	Información de cada evento
5	Participantes	Personas que asisten
6	Inscripciones	Participación en eventos
7	Pagos	Pagos realizados

1. CREAR LA BASE DE DATOS

Este código está preparado para **SQL Server**:

SQL

```
CREATE DATABASE PlataformaEventos;  
GO
```

```
USE PlataformaEventos;  
GO
```



2. TABLA CATEGORIAS

SQL

```
CREATE TABLE Categorias (  
    id_categoria INT IDENTITY(1,1) PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    descripcion VARCHAR(250)  
);  
GO
```

Ejemplos:

1		Concierto
2		Taller
3		Conferencia
4		Festival
5		Cultura



3. TABLA LUGARES

SQL

```
CREATE TABLE Lugares (  
    id_lugar INT IDENTITY(1,1) PRIMARY KEY,  
    nombre VARCHAR(150) NOT NULL,  
    ciudad VARCHAR(100) NOT NULL,  
    capacidad INT NOT NULL,  
    direccion VARCHAR(200)  
);  
GO
```



4. TABLA ORGANIZADORES

</> SQL

```
CREATE TABLE Organizadores (
    id_organizador INT IDENTITY(1,1) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    telefono VARCHAR(20),
    correo VARCHAR(150)
);
GO
```



5. TABLA EVENTOS

Esta es la tabla central.

</> SQL

```
CREATE TABLE Eventos (
    id_evento INT IDENTITY(1,1) PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    descripcion VARCHAR(300),
    fecha DATE NOT NULL,
    hora TIME NOT NULL,
    precio DECIMAL(10,2) NOT NULL,

    id_categoria INT NOT NULL,
    id_lugar INT NOT NULL,
    id_organizador INT NOT NULL,

    CONSTRAINT FK_Evento_Categoria
        FOREIGN KEY (id_categoria)
        REFERENCES Categorias(id_categoria),

    CONSTRAINT FK_Evento_Lugar
        FOREIGN KEY (id_lugar)
        REFERENCES Lugares(id_lugar),

    CONSTRAINT FK_Evento_Organizador
        FOREIGN KEY (id_organizador)
        REFERENCES Organizadores(id_organizador)
);
GO
```

Aquí podemos explicar algo muy importante:

Eventos

```
|
├─ id_categoria
├─ id_lugar
└─ id_organizador
```

Esas columnas son **claves foráneas**.



6. TABLA PARTICIPANTES

SQL

```
CREATE TABLE Participantes (
    id_participante INT IDENTITY(1,1) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    documento VARCHAR(20) UNIQUE NOT NULL,
    telefono VARCHAR(20),
    correo VARCHAR(150)
);
GO
```

7. TABLA INSCRIPCIONES

Esta tabla conecta:

PARTICIPANTE

+

EVENTO

↓

INSCRIPCIÓN

SQL

```
CREATE TABLE Inscripciones (
    id_inscripcion INT IDENTITY(1,1) PRIMARY KEY,
    fecha_inscripcion DATE NOT NULL DEFAULT GETDATE(),
    estado VARCHAR(30) NOT NULL,

    id_participante INT NOT NULL,
    id_evento INT NOT NULL,

    CONSTRAINT FK_Inscripcion_Participante
        FOREIGN KEY (id_participante)
        REFERENCES Participantes(id_participante),

    CONSTRAINT FK_Inscripcion_Evento
        FOREIGN KEY (id_evento)
        REFERENCES Eventos(id_evento)
);
GO
```

8. TABLA PAGOS

SQL

```
CREATE TABLE Pagos (
    id_pago INT IDENTITY(1,1) PRIMARY KEY,
    fecha_pago DATE NOT NULL DEFAULT GETDATE(),
    valor DECIMAL(10,2) NOT NULL,
    metodo_pago VARCHAR(50) NOT NULL,
    estado VARCHAR(30) NOT NULL,

    id_inscripcion INT NOT NULL,

    CONSTRAINT FK_Pago_Inscripcion
        FOREIGN KEY (id_inscripcion)
        REFERENCES Inscripciones(id_inscripcion)
);
GO
```



9. INSERTAR DATOS

Para que los estudiantes puedan probar los procedimientos:

Categorías

```
<> SQL

INSERT INTO Categorías (nombre, descripcion)
VALUES
('Concierto', 'Eventos musicales'),
('Taller', 'Actividades prácticas'),
('Conferencia', 'Eventos académicos'),
('Festival', 'Eventos culturales'),
('Experiencia', 'Experiencias recreativas');
GO
```

Lugares

```
<> SQL

INSERT INTO Lugares (nombre, ciudad, capacidad, direccion)
VALUES
('Teatro Central', 'Medellín', 500, 'Carrera 45 #10-20'),
('Centro Cultural Norte', 'Medellín', 300, 'Calle 50 #30-15'),
('Auditorio Principal', 'Bogotá', 800, 'Carrera 10 #20-30'),
('Plaza Cultural', 'Cali', 1000, 'Calle 5 #15-20');
GO
```

Organizadores

```
<> SQL

INSERT INTO Organizadores
(nombre, apellido, telefono, correo)
VALUES
('Carlos', 'Ramírez', '3001112233', 'carlos@email.com'),
('Laura', 'Gómez', '3012223344', 'laura@email.com'),
('Andrés', 'Martínez', '3023334455', 'andres@email.com');
GO
```

Eventos

```
<> SQL

INSERT INTO Eventos
(nombre, descripcion, fecha, hora, precio,
 id_categoria, id_lugar, id_organizador)
VALUES
('Noche de Rock', 'Concierto de bandas locales',
 '2026-10-15', '19:00', 85000, 1, 1, 1),

('Taller de Fotografía', 'Taller básico de fotografía',
 '2026-10-20', '14:00', 45000, 2, 2, 2),

('Tecnología del Futuro', 'Conferencia sobre tecnología',
 '2026-11-05', '09:00', 30000, 3, 3, 3),

('Festival Cultural', 'Festival de música y cultura',
 '2026-11-20', '16:00', 60000, 4, 4, 1);
GO
```

Participantes

<> SQL

```

INSERT INTO Participantes
(nombre, apellido, documento, telefono, correo)
VALUES
('Daniel', 'López', '1001', '3101111111', 'daniel@email.com'),
('María', 'Torres', '1002', '3102222222', 'maria@email.com'),
('Juan', 'Pérez', '1003', '3103333333', 'juan@email.com'),
('Sofía', 'García', '1004', '3104444444', 'sofia@email.com');
GO

```

AHORA VIENE LO INTERESANTE

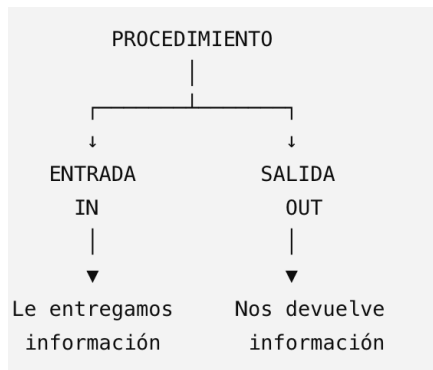
PROCEDIMIENTOS ALMACENADOS

Explícales:

Un procedimiento almacenado es como una función que vive dentro de SQL Server.

Puede recibir información, trabajar con ella y devolver información.

Tenemos:



PROCEDIMIENTO 1

Parámetro de ENTRADA

Queremos buscar eventos de una determinada categoría.

<> SQL

```

CREATE PROCEDURE sp_BuscarEventosCategoria
    @id_categoria INT
AS
BEGIN
    SELECT
        e.id_evento,
        e.nombre,
        e.fecha,
        e.hora,
        e.precio
    FROM Eventos e
    WHERE e.id_categoria = @id_categoria;

END;
GO

```

Lo ejecutamos:

```
</> SQL  
  
EXEC sp_BuscarEventosCategoria 1;
```

Aquí:

```
1  
↓  
@id_categoria  
↓  
SQL Server busca  
↓  
Eventos de categoría 1
```

PROCEDIMIENTO 2

Parámetro de SALIDA

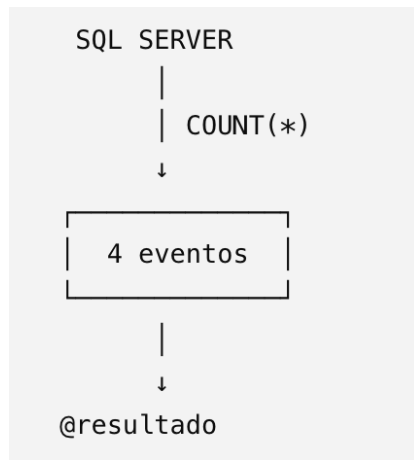
Ahora queremos saber **cuántos eventos existen**.

```
</> SQL  
  
CREATE PROCEDURE sp_CantidadEventos  
    @cantidad INT OUTPUT  
AS  
BEGIN  
  
    SELECT @cantidad = COUNT(*)  
    FROM Eventos;  
  
END;  
GO
```

Para ejecutarlo:

```
</> SQL  
  
DECLARE @resultado INT;  
  
EXEC sp_CantidadEventos @resultado OUTPUT;  
  
SELECT @resultado AS CantidadEventos;
```

La idea visual:



PROCEDIMIENTO 3

ENTRADA + SALIDA

Este es el que yo utilizaría como ejercicio principal.

Queremos consultar cuántas personas están inscritas en un evento.

```
</> SQL

CREATE PROCEDURE sp_ContarInscritos
    @id_evento INT,
    @cantidad INT OUTPUT
AS
BEGIN

    SELECT @cantidad = COUNT(*)
    FROM Inscripciones
    WHERE id_evento = @id_evento;

END;
GO
```

Ejecutamos:

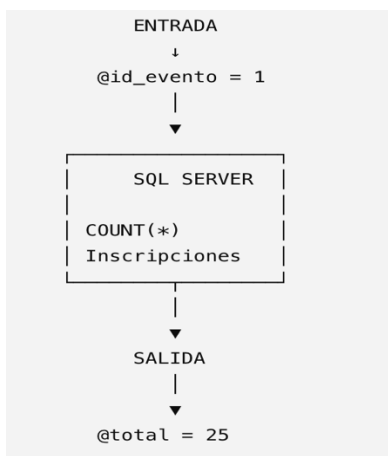
```
</> SQL

DECLARE @total INT;

EXEC sp_ContarInscritos
    @id_evento = 1,
    @cantidad = @total OUTPUT;

SELECT @total AS TotalInscritos;
```

Aquí ocurre algo espectacular para explicar:



RETO FINAL PARA LOS ESTUDIANTES

"El sistema de eventos necesita un panel administrativo"

Crear los siguientes procedimientos:

Reto 1 — Entrada

Crear:

sp_BuscarEvento

Debe recibir:

@id_evento

y mostrar:

- nombre
- fecha
- hora
- precio
- lugar
- categoría

Reto 2 — Salida

Crear:

sp_TotalParticipantes

Debe devolver mediante OUTPUT la cantidad total de participantes.
Reto 3 — Entrada + Salida

Crear:

sp_CalcularRecaudoEvento

Debe recibir:

@id_evento

y devolver:

@recaudo OUTPUT

La fórmula será:

$\text{recaudo} = \text{participantes} \times \text{precio del evento}$

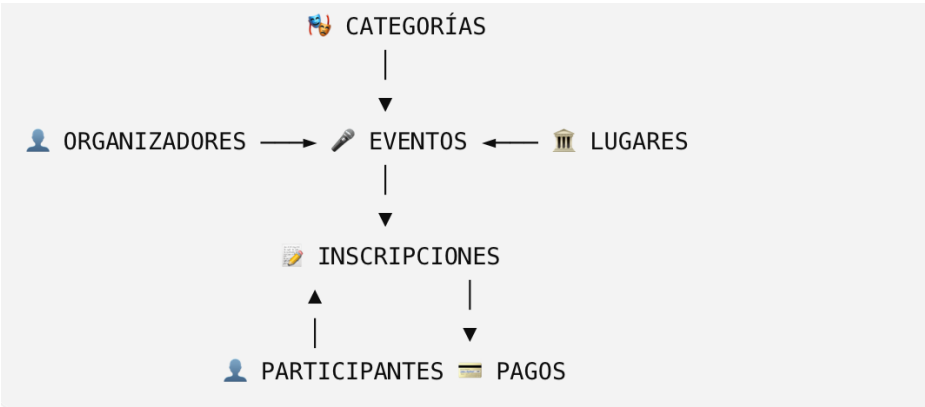
Por ejemplo:

Evento: Noche de Rock
Precio: \$85.000
Participantes: 20

20×85.000

$= \$1.700.000$

Una base de datos no es solamente guardar información. Un buen desarrollador consigue que la base de datos piense, consulte, calcule y responda. Hoy ustedes no solamente crearon 7 tablas: construyeron un pequeño sistema capaz de recibir información y devolver resultados.



7 tablas + relaciones + INSERT + SELECT + procedimientos + parámetros IN + parámetros OUTPUT + cálculos.